Dynamic Inventory Risk & Reorder Engine: An End-to-End Operational Analytics Pipeline

Jacob C Sellinger

Northwest Missouri State University, Maryville MO 64468, USA
S577258@nwmissouri.edu

Abstract. WORK IN PROGRESS
This capstone project presents an end-to-end data engineering and analytics pipeline designed to optimize supply chain inventory under lead-time uncertainty. By processing raw transactional data using DuckDB, predicting supplier lead times with machine learning, and calculating dynamic safety stock levels, the engine surfaces real-time stockout risks via an interactive Python Streamlit dashboard.

Keywords: Supply Chain Analytics · Lead Time Prediction · Dynamic Reorder Point · DuckDB · Streamlit.

1 Introduction

WORK IN PROGRESS

Modern supply chains face significant volatility in lead times and demand. Traditional static reorder point models often fail under stochastic conditions, resulting in costly stockouts or excessive holding costs...

2 Methodology Overview

WORK IN PROGRESS

The system architecture consists of a three-stage pipeline: ingestion and cleaning (ETL), predictive lead time modeling, and interactive decision support...

3 Data Collection

3.1 Dataset Overview

The dataset that will be used in this project is version 5 of the *DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS* from Mendeley Data. [2] This data set is a comprehensive supply chain data set that covers procurement, production, sales, and commercial distribution. It comes from DataCo Global, a



Fig. 1. End-to-End Supply Chain Data Pipeline and Web Deployment Swimlane.

UK-based consultancy company, that provides data and information management consultancy. This specific data set is publicly available for supply chain projects and is well documented, which allows easy implementation.

This paper will use the standard ETL, as opposed to ELT, and follow the basic steps: Extract, Transofrm, and Load. ETL will be easier to load to our database and keep the project manageable by mitigating scale and complexity. It also allows us to cleanly normalize our data, even though we are already using a structured data set, we will still want to do basic transformations and drop any unnecessary columns or rows.

3.2 Data Extraction

Extraction requires us to pull the data from a non-local source, in this case we will be pulling from kaggle using the kaggle API given that it is well documented and easy to use, while demonstrating rudimentary knowledge of the API calls and non-local extraction.

The main coding language used in this paper will be Python which gives further credence to using the kaggle API, kagglehub. This will help build a modular pipeline that is dataset agnostic and can be used as needed in the future for any kaggle datasets. The API is already designed to work seamlessly in Python NML workflows and it is free to create personal API Tokens. [4]

To create a manageable and more thorough documentation we will use `uv` to add all dependencies and packages for this project.

```
#uv add kaggle hub
uv add kagglehub

#examples of other added dependencies, please refer to the uv lock or README for all
uv add pandas
uv add sci-kit
uv add skilearn

```

Listing 1.1. Adding kagglehub with uv

Once installed, we will load the dataset directly to a Pandas DataFrame. The documentation on kaggle gives a good overview of how to perform this, we will be slightly modifying kaggle's example to handle.

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Load a DataFrame with a specific version of a CSV
df = kagglehub.dataset_load(
KaggleDatasetAdapter.PANDAS,
"shashwatwork/dataco-smart-supply-chain-for-big-data-analysis",
```

```
    "DataCoSupplyChainDataset.csv",
    )

```

Listing 1.2. Loading Kaggle CSV dataset into a Pandas DataFrame

4 Pre-Processing and Storage

4.1 Data Transformation

Transformation will be the initial setup of our data and we should ideally manipulate the data in such a way that it is read to load into our database or be directly used in our ML pipeline. Transformation is less important when working with small datasets, as we are less limited by memory or CPU bottlenecks, hence why the entire dataset will be loaded. If we had pursued an ELT method we could have pulled in only what we needed and then manipulated it but for visibility into the reasoning for certain transformations, choices, and better data exploration we are making a deliberate choice to do pull the entire dataset and perform transformations on the entire set or as needed. Given that information we will perform minimal transformations beyond standard data cleaning and normalization by droppping missing critical values, which are those that we are using or targeting, explicitly type cast certain variables, and any feature engineering that may be necessary.

We will want to drop rows that are missing only non-critical values. For example, we are targeting lead time for our ML algorithm so we want to avoid dropping this unnecessarily or any of the attributes that we believe we might use. In this case, from my real world experience, we will likely use Shipping Mode, Shipping Date, Order City, Order Country, and Category_ID are likely to help us train an ML model to predict Days for Shipping (real).

```
    # Define the critical columns needed for your ML model
    critical_cols = [
        'Shipping Mode',
        'shipping date (DateOrders)',
        'Order City',
        'Order Country',
        'Category Id',
        'Days for shipping (real)'  # Your target variable
    ]

    # Drop rows missing values in ANY of these key attributes
    df = df.dropna(subset=critical_cols)

```

Listing 1.3. Dropping rows with missing values in key feature columns

After dropping critical rows that have null values we will want to type set anything so that everything is normalized. For example, dates and times are

passed as strings instead of true date times so we will need to type set shipping date (DateOrders)

```
1
2     # Load the DataCo Supply Chain dataset
3     df = kagglehub.dataset_load(
4         KaggleDatasetAdapter.PANDAS,
5         "shashwatwork/dataco-smart-supply-chain-for-big-data-
      analysis",
6         "DataCoSupplyChainDataset.csv",
7     )
8
9     # Define critical columns needed for target and ML
      features
10    critical_cols = [
11        'Shipping Mode',
12        'shipping date (DateOrders)',
13        'Order City',
14        'Order Country',
15        'Category Id',
16        'Days for shipping (real)'  # Target variable
17    ]
18
19    # Drop rows with missing values strictly within critical
      columns
20    df = df.dropna(subset=critical_cols)
21
22    # Explicit Type Casting (Datetime Conversion)
23    date_cols = ['order date (DateOrders)', 'shipping date (
      DateOrders)']
24    for col in date_cols:
25        df[col] = pd.to_datetime(df[col])
26
27    # Type Casting Categorical Columns
28    cat_cols = [
29        'Shipping Mode',
30        'Order Country',
31        'Order City',
32        'Category Id',
33        'Type',
34        'Delivery Status'
35    ]
36    df[cat_cols] = df[cat_cols].astype('category')
37
```

Listing 1.4. Data preprocessing, type casting, and targeted cleaning

4.2 Feature Engineering

We will perform some basic feature engineering now that we done some basic normalization of the dataset. Feature engineering for ML will allow us to test different types of ML algorithms. Most standard Skikit-Learn models cannot use categorical variables so we need to engineer a column so that they are categorical. Examples of the Scikit-Learn models that we might use are regression, random forest, and decision trees. ML models that I have not explicitly used but that can take categorical variables natively are modles that convert them to integers on the fly such as LightGBM or CatBoost. We will not be using or testing either of these for this project.

We will show an example of turning the variable 'Shipping Mode' into an ordinal variable rather than categorical but we can apply the same method to all categorical variables. Turning categorical data into ordinal data does have some rules such as importance where the relationship is meaningful. [3] In the below case, we assigned it based on priority.

```
    # Define ordinal ranking dictionary (higher number =
    faster priority)
    shipping_mode_order = {
        'Standard Class': 1,
        'Second Class': 2,
        'First Class': 3,
        'Same Day': 4
    }

    # Apply ordinal mapping to create a new engineered
    feature
    df['Shipping_Mode_Ordinal'] = df['Shipping Mode'].map(
    shipping_mode_order)

    # Verify mapping result
    print(df[['Shipping Mode', 'Shipping_Mode_Ordinal']].
    drop_duplicates())

```

Listing 1.5. Ordinal encoding of Shipping Mode based on delivery priority

As an important note, we have explicitly said "ordinal" but categorical data can also be turned into "nominal" where the priority or importance does not matter. SciKit also comes with some built in encoders to transform items for us should we not want to assign every variable ourselves. [5]

4.3 Data Loading

STILL WORKING OUT KINKS

Data loading is our final step where we can take all transformations and keep them in a static state that we can then train our ML model on our interact with how we see fit. We will be using DuckDB given that it has seamless integration

with pandas dataframes which allows us to embed it directly into DuckDB for a serverless layer. This should help us in end web deployment with streamlit.

We will first need to create the DuckDB database. This is a relational database [?]

```
db_path = "dataco_supply_chain.db"

```

Listing 1.6. ETL Loading Phase: DuckDB Database Creation

Then we can make our connection to the DuckDB database and load in our dataframe.

```

# Establish a connection to a local DuckDB database file
# (Use ':memory:' for an in-memory database instance)
con = duckdb.connect('dataco_supply_chain.duckdb')

# Register and load the cleaned Pandas DataFrame into DuckDB as a table
# DuckDB directly queries and ingests Pandas DataFrames without disk I/O overhead
con.execute("CREATE OR REPLACE TABLE supply_chain_events AS SELECT * FROM df")

# Verify successful table creation and record counts
row_count = con.execute("SELECT COUNT(*) FROM supply_chain_events").fetchone()[0]
print(f"Successfully loaded {row_count:,} records into DuckDB table 'supply_chain_events'.")

# Close the connection when done - This is not necessary but good practice
con.close()

```

Listing 1.7. ETL Loading Phase: Ingestion into DuckDB

5 Data Validation

Following pre-processing and persistence in DuckDB, we execute an automated data validation routine prior to model training. This step verifies data integrity, schema consistency, and matrix readiness directly from the database connection.

5.1 Post-Storage Verification

Check if pandas df was properly stored and persistent

5.2 Constraint and Attribute Type Checks

Check if attributes and variables, such as lead time and contractor, are in valid typesetting and feature engineered appropriately.

5.3 Matrix Integrity and Non-Null Guarantees

Quick check to ensure all zero, missing or null values were dropped and maintained in the database for training

6 Predictive Modeling: Random Forest Regression

6.1 ML Model Selection and Rationale

6.2 ML Model Setup and Training

This will cover how I trained the machine learning model, the type of model, how it was trained, the rationale for the model chosen, and whether or not there are limitations to my choice and rationale

6.3 ML Model Evaluation

This will be the evaluation of my model to find specific limitations so that we can make inferences about when this model may fail, whether it be on edge cases or is even useful for deployment

7 Web Deployment

7.1 Dashboard Development

To cover an E2E pipeline, we would need a way to interact with the model through some form of GUI - that way, the end-user can interact or at least see the data in a helpful visual format. In essence, the degree is more about analysis and presentation of data rather than my ability to code, so I view this as a vital part. This may also have another subsection about how the dashboard will be deployed

7.2 Interactive Dashboard Development

This will be a rather small section, but because this will likely be done piecemeal and modular, there will need to be some time spent on integration and to give a good overview of how the parts actually connect

7.3 System Integration and Web Hosting

I believe a final, perhaps optional step, would be to deploy the project to the web - I would likely lack live-streamed data, but from an end-to-end view covering some form of hosting and deployment will be beneficial from a knowledge point of view and demonstration of skills.

8 Interpretation and Analysis of Results

This is the final part of the body paragraph where I will examine the results of my project and analyze how well it performs; we might simply perform an ad hoc evaluation or give it edge cases to see where it fails

9 Conclusion

This is the conclusion, broadly summarizing the work that was completed, the results, and analysis. This section should also at least have suggestions for more room for improvement, as that segue's directly into our limitations.

10 Limitations

While I will likely give limitations throughout the paper, having a limitations section at the end of the paper can summarize the found limitations and any other proposed larger limitations

[1]

References

1. Bandi, A.: Operational data pipelines in modern supply chains. Journal of Data Analytics **12**(2), 45–58 (2026)
2. Constante, F., Silva, F., Pereira, A.: DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS (2019). https://doi.org/10.17632/8gx2fvg2k6.5
3. GeeksforGeeks: Categorical Data Encoding Techniques in Machine Learning. https://www.geeksforgeeks.org/machine-learning/categorical-data-encoding-techniques-in-machine-learning/ (2026)
4. Kaggle: How to Use Kaggle. https://www.kaggle.com/docs/api (2026)
5. Learn, S.: OrdinalEncoder. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html (2026)